

**University of Science and
Technology
- Wrocław-**



**LABORATORY 4: UTILIZING SOFTWARE
FOR SYSTEM MODELLING**

**SUPPORT FOR IT PROJECT MANAGEMENT
2025/2026**

Teacher: Piotr Zabawa

Student: Álvaro Puebla Ruisánchez 293867

SUPPORT FOR IT PROJECT MANAGEMENT REPORT

Table of Contents

Table of Contents	2
1. Software Overview	3
1.1 Name and Purpose	3
1.2 Key Functionality	3
1.3 Why Lucidchart Was Selected	3
2. Performed Tasks	4
2.1 Task 1 - Basic Structure & Actors	4
2.2 Task 2 - Connections & Formatting	5
2.3 Task 3 - Your own system modelling	6
3. Summary	7
3.1 Software Suitability Assessment	7
3.2 Evaluation of Software Functionality	7
3.3 User Interface Evaluation	7
3.4 Evaluation of Other Software Attributes	8

SUPPORT FOR IT PROJECT MANAGEMENT REPORT

1. Software Overview

1.1 Name and Purpose

The software selected for this laboratory session is **Lucidchart**, a web-based diagramming and visual communication application developed by Lucid Software Inc. Its primary purpose is to enable users to create, share, and collaborate on professional diagrams, including UML models, flowcharts, entity-relationship diagrams, and system architecture blueprints. In the context of IT project management, Lucidchart serves as a requirements modelling tool that translates complex system specifications into clear visual representations, facilitating communication between technical and non-technical stakeholders throughout the software development lifecycle.

1.2 Key Functionality

Lucidchart provides a comprehensive set of features oriented towards system modelling and collaborative documentation. Its core functionalities include a standardised UML shape library compliant with UML 2.5, which ensures that all diagrams follow industry-standard notation. The tool features intelligent relationship anchoring, whereby connector lines between elements remain logically attached when components are repositioned on the canvas, preserving the integrity of the model. Real-time co-authoring allows multiple users to edit the same document simultaneously, with changes reflected live across all sessions. Additionally, Lucidchart supports multi-format export (PDF, PNG, SVG), cloud storage synchronisation via Google Drive and OneDrive, a contextual commenting system for in-diagram feedback, and an extensive library of pre-built templates covering UML, network infrastructure, and process engineering diagrams.

1.3 Why Lucidchart Was Selected

The selection of Lucidchart followed a structured market analysis comparing four candidate tools: Enterprise Architect (Sparx Systems), Visual Paradigm, Draw.io, and Lucidchart. The evaluation was driven by a set of defined project requirements: cloud-based accessibility, real-time collaboration, an intuitive drag-and-drop interface, export capabilities, and budget constraints. Enterprise Architect was ruled out due to its high cost (\$245 one-time licence) and complexity, which exceeded the needs of an agile academic team. Visual Paradigm, while feature-rich, presented a cluttered interface that posed usability challenges for beginners. Draw.io, although fully free and open-source, lacked robust team management features and exhibited instability in its real-time collaboration functionality. Lucidchart, offering a free tier with up to three editable documents, a highly polished interface, seamless real-time collaboration, and a vast template library, was determined to provide the best balance of functionality and cost for the immediate project needs.

SUPPORT FOR IT PROJECT MANAGEMENT REPORT

2. Performed Tasks

2.1 Task 1 - Basic Structure & Actors

Task description:

The first task consisted of constructing the foundational structural elements of a UML Use Case Diagram for a Library Management System using Lucidchart. The process began by navigating to www.lucidchart.com, signing up for a free account, and creating a new blank document via the "+ New" > "Lucidchart" > "Blank Document" pathway. Once on the editing canvas, a UML System Boundary rectangle was inserted from the shape library and labelled "Library Management System." This boundary box visually delimits the scope of the system under analysis.

Subsequently, three Actor elements - represented by stick figures - were placed outside the system boundary: "Student" on the left side, and "Librarian" and "DB System" on the right side. These actors represent the external entities that interact with the system. Inside the boundary, four Use Case elements (oval shapes) were added and labelled: "Search Book," "Borrow Book," "Return Book," and "Update Catalog." At this stage, no connections were drawn between actors and use cases; the focus was exclusively on placing and naming all structural components correctly.

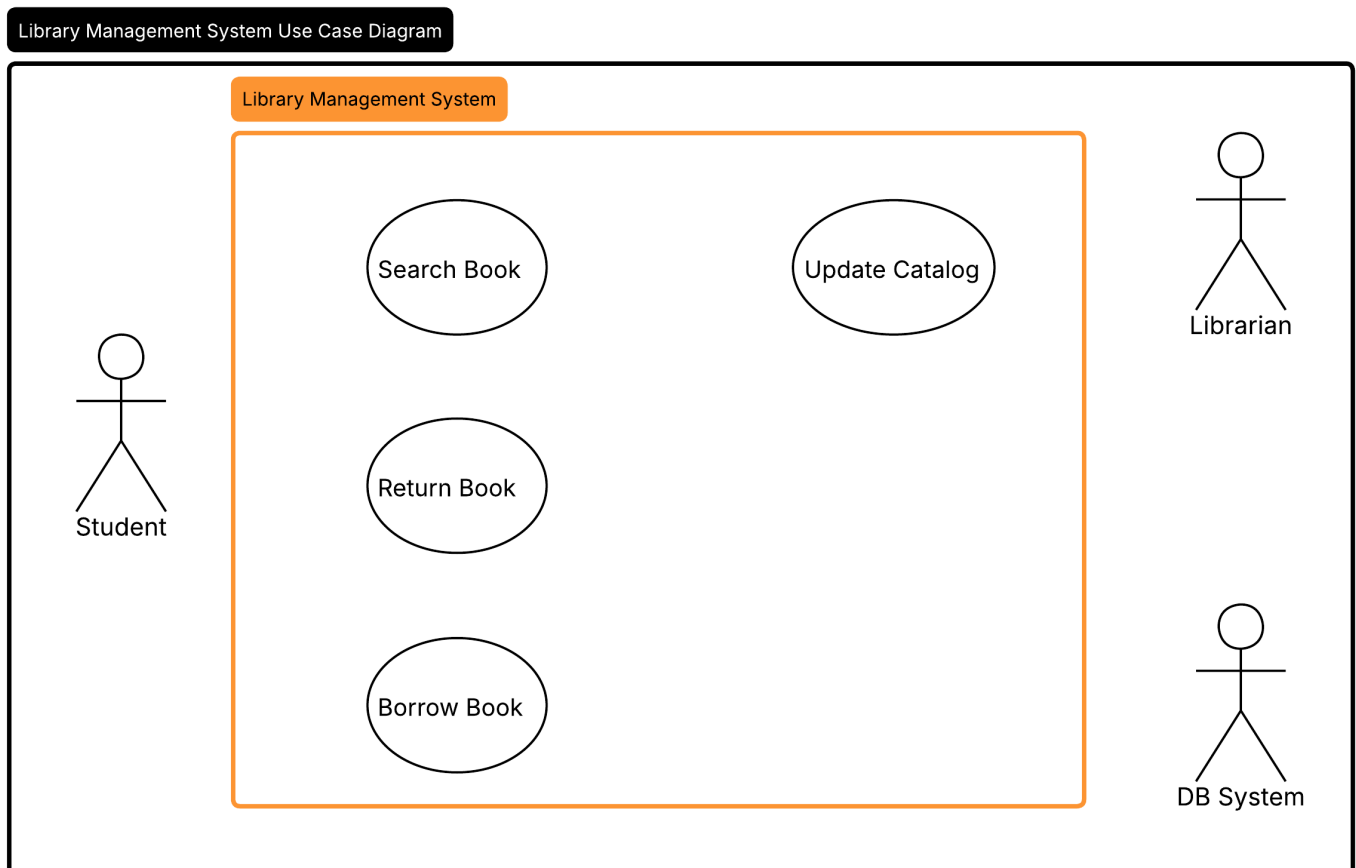


Figure 1. Basic structure of the Library Management System Use Case Diagram, showing the system boundary, actors, and unconnected use cases as created in Lucidchart.

SUPPORT FOR IT PROJECT MANAGEMENT REPORT

2.2 Task 2 - Connections & Formatting

Task description:

Building upon the structure established in Task 1, the second task focused on adding association lines between actors and use cases, and applying visual formatting to improve the diagram's clarity and readability. Association lines (solid lines representing actor-to-use-case interactions) were drawn as follows: the "Student" actor was connected to "Search Book," "Borrow Book," and "Return Book," reflecting the three primary operations available to library users. The "Librarian" actor was connected to "Update Catalog," representing the administrative function exclusive to that role. All four use cases - "Search Book," "Borrow Book," "Return Book," and "Update Catalog" - were connected to the "DB System" actor, indicating that every operation involves interaction with the underlying database system.

Following the connection phase, visual formatting was applied. All Actor figures were recoloured to blue, and all Use Case oval shapes were filled with yellow. This colour coding enhances diagram legibility by allowing readers to instantly distinguish between actor types and system actions at a glance.

Library Management System Use Case Diagram

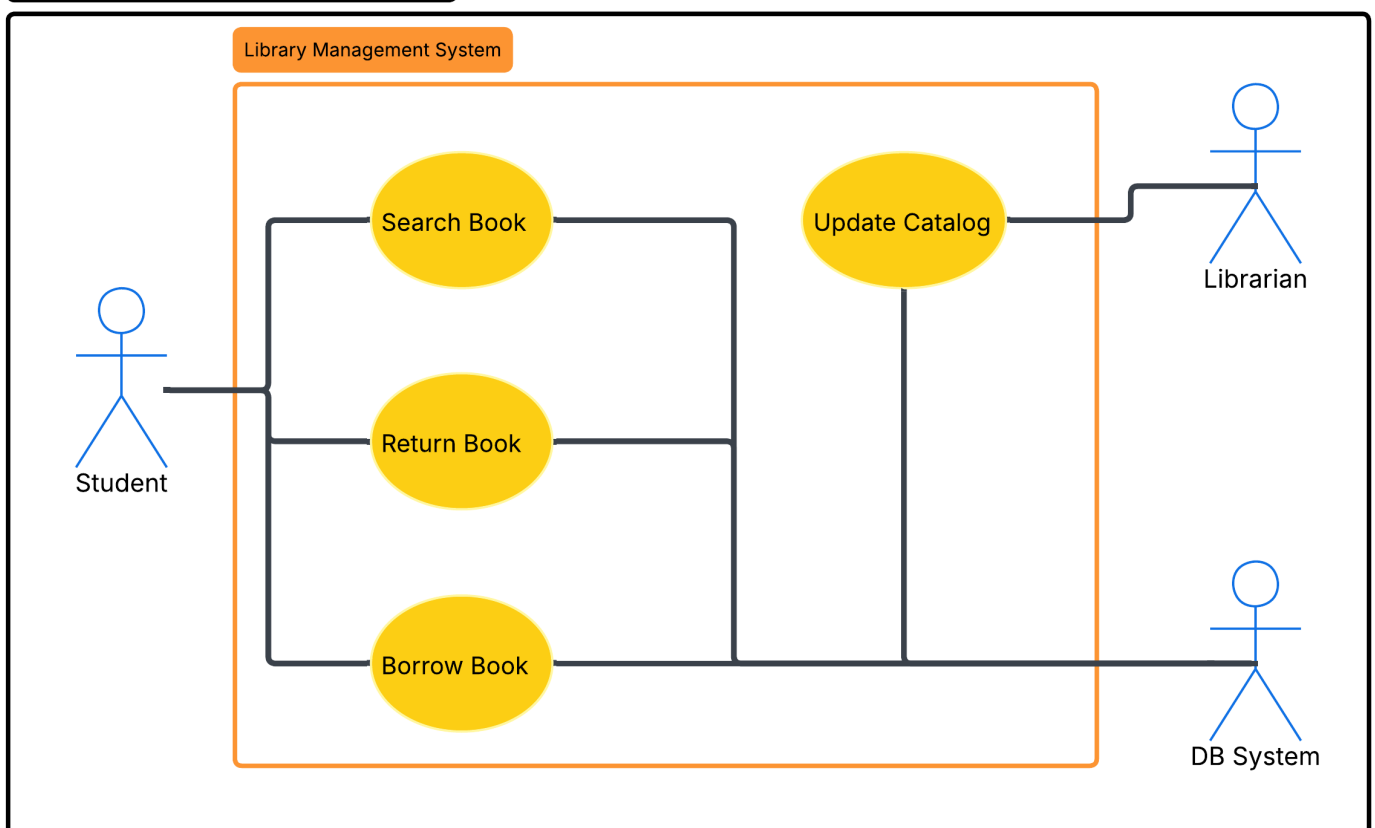


Figure 2. Completed Library Management System Use Case Diagram with all actor-to-use-case associations drawn and colour formatting applied (blue actors, yellow use cases).

SUPPORT FOR IT PROJECT MANAGEMENT REPORT

2.3 Task 3 - Your own system modelling

Task description:

For the third task, an original UML Use Case Diagram was designed from scratch, modelling a **Resource Management System** - a system representative of tools used in academic or institutional environments for booking and managing shared physical resources such as lab equipment, study rooms, or library materials.

The diagram defines a "Reservation System" as the system boundary. Two actors are identified: "Student," positioned on the left as the primary user, and "Researcher," positioned at the lower left as a specialised user with an inheritance relationship to Student (denoted by a generalisation arrow). Inside the system boundary, six use cases are defined: "Search Resources," "Make Reservation," "Scan QR," "View Availability," "Return Resource," and "Check Out Resource." The Student actor is connected to "Search Resources," "Make Reservation," "Scan QR," and "View Availability." The Researcher actor is connected to "Schedule Recurring Reservation," and inherits the Student's connections through the generalisation relationship.

Two extend relationships are modelled using dashed arrows: "Return Resource" extends "Scan QR," and "Check Out Resource" also extends "Scan QR," indicating that scanning a QR code is the base interaction that can optionally trigger either a resource checkout or return action. The colour scheme follows the convention established in Task 2: actors are rendered in blue and use case ovals in yellow, ensuring visual consistency.

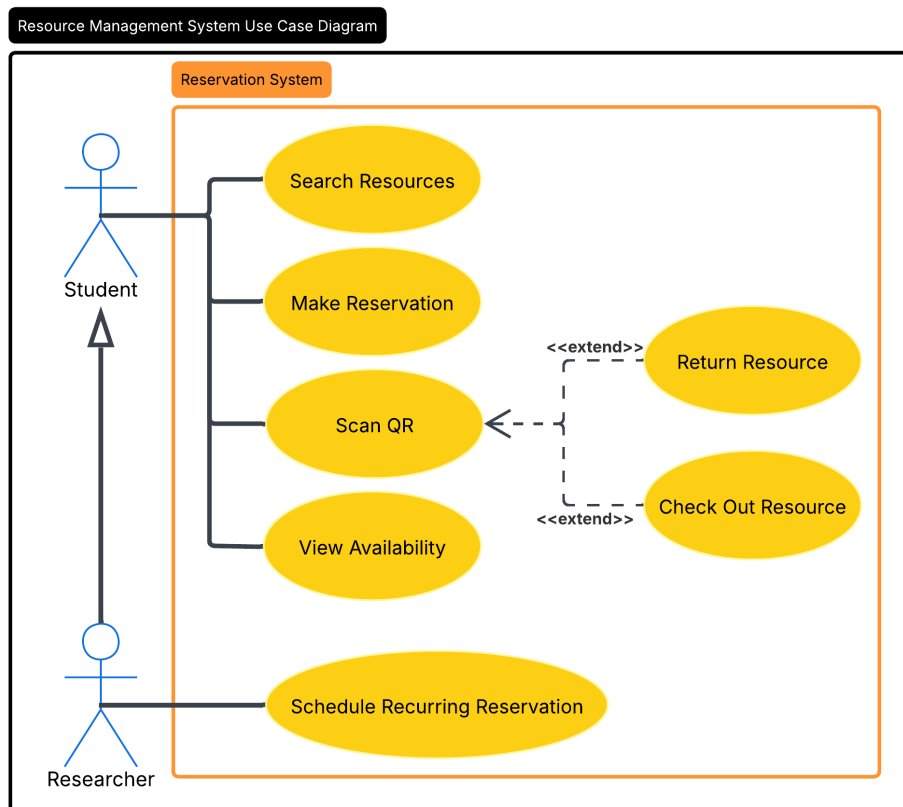


Figure 3. Resource Management System Use Case Diagram, modelled independently, featuring two actors, six use cases, generalisation and extend relationships, and colour-coded formatting.

SUPPORT FOR IT PROJECT MANAGEMENT REPORT

3. Summary

3.1 Software Suitability Assessment

Lucidchart demonstrates a high degree of suitability for requirements management and system modelling tasks in the context of IT project management. Its support for standardised UML notation ensures that diagrams produced are compatible with widely accepted industry practices, facilitating communication between project managers, software architects, and clients. The tool's real-time collaboration capability directly addresses one of the most common bottlenecks in distributed project teams - the synchronisation of design decisions. Furthermore, its cloud-native architecture eliminates the friction associated with version management of locally stored files. For the specific tasks performed in this laboratory - constructing, refining, and independently designing UML Use Case Diagrams - Lucidchart proved entirely adequate, covering all required modelling primitives (system boundaries, actors, use cases, associations, extend relationships, and generalisation) without requiring any premium features.

3.2 Evaluation of Software Functionality

The functionality offered by Lucidchart is both broad and well-integrated. The UML shape library is comprehensive, providing all standard UML 2.5 elements within an easily searchable panel. The intelligent anchoring of connector lines significantly reduces the effort required to maintain diagram consistency when elements are reorganised. The ability to apply custom colours to individual shapes, as exercised during the formatting task, is straightforward and immediate. The export functionality performed reliably, producing high-resolution PNG outputs suitable for academic documentation. One notable limitation of the free tier is the restriction to a maximum of three editable documents, which may constrain productivity in larger or longer-running projects without a paid subscription. The absence of version history on the free plan is also a limitation, as it prevents users from reverting to earlier diagram states in case of errors.

3.3 User Interface Evaluation

Lucidchart's user interface is clean, modern, and well-organised. The left-side shape panel, top formatting toolbar, and central canvas follow a layout pattern consistent with contemporary productivity tools, reducing the initial learning curve for new users. Drag-and-drop interaction is responsive, and snap-to-grid alignment assists in producing well-structured diagrams with minimal manual adjustment. Labels can be edited by double-clicking directly on shapes, which is intuitive. The contextual formatting options that appear upon element selection - covering fill colour, border style, font, and line type - are logically positioned and accessible without navigating nested menus. Overall, the interface strikes an effective balance between simplicity for occasional users and depth of options for experienced modellers.

SUPPORT FOR IT PROJECT MANAGEMENT REPORT

3.4 Evaluation of Other Software Attributes

From a performance perspective, Lucidchart operates with low latency on standard hardware and a stable internet connection, with no observable lag during shape manipulation or diagram rendering throughout the session. Accessibility is a notable strength: as a web application, Lucidchart requires no installation and is accessible from any modern browser on any operating system, making it particularly well-suited for cross-platform academic environments. The platform's integration with Google Drive and Microsoft OneDrive further enhances accessibility by enabling seamless document retrieval across devices. In terms of scalability, while the free tier is sufficient for introductory tasks, the pricing model (\$7.95/month for Individual Premium) is reasonable for professional or extended academic use. One constraint worth noting is that offline usage is not supported, meaning an active internet connection is mandatory - a potential limitation in environments with unreliable connectivity.